

# TECHNICAL DOCUMENTATION

## Project 1: Simple Rule-Based ChatBot System

**Intern Name:** Sake Nishanth  
**Domain:** Artificial Intelligence  
**Organization:** DecodeLabs  
**Development Stack:** Python 3 (Standard Library)

### 1. Introduction & Theoretical Framework

In the computational paradigm of Natural Language Processing (NLP), conversational agents are classified into two overarching domains: generative architectures and rule-based systems. This project explores the foundational engineering of a **Deterministic Rule-Based ChatBot**. Rule-based frameworks act as the bedrock of automation, utilizing exact key-value mapping logic to resolve high-frequency, low-variance human interactions without incurring the stochastic volatility or high latency costs associated with neural text generation.

The scope of this implementation encompasses three core structural constraints outlined by the project parameters: utilizing native hash-maps via Python dictionaries, managing an persistent execution cycle via an infinite event loop, and processing inputs against clean, predefined message indices.

### 2. System Architecture & Core Mechanics

The technical topology of the system operates on a linear, state-free processing pipeline. When a user transmits an arbitrary textual sequence, the runtime evaluates the input according to the following mathematical lookup strategy:

Let  $M$  be the message bank structured as a dictionary of key-value pairs, where  $k \in K$  represents the standardized user intent and  $v \in V$  represents the static system dispatch response. The input string  $x$  provided by the user is normalized through a deterministic purification function  $f(x)$  such that:

$$f(x) = \text{ext\{lower\}}(\text{ext\{strip\}}(x))$$

The decision evaluation boundary resolves to:

$$\text{ext\{Response\}}(x) = \text{egin\{cases\}} M[f(x)], \& \text{ext\{if\}} f(x) \notin K \text{ ext\{Fallback Response\}}, \& \text{ext\{if\}} f(x) \notin K \\ \text{end\{cases\}}$$

### 3. Structural Code Analysis

The system initialises by creating an in-memory hash-map. The utilization of standard dictionary structures provides a time complexity of  $O(1)$  for retrieval operations, making it extremely scalable in low-latency runtime applications.

```
message_bank = {
    "hi": "Hello!",
    "hello": "Hello! How can I help you today?",
    "good morning": "Good Morning!",
    "good afternoon": "Good Afternoon!",
    "good evening": "Good Evening!",
    "who are you": "I am a Rule-Based AI Chatbot.",
    "what is your name": "I am a Chatbot.",
    "thank you": "You're welcome!",
    "bye": "Goodbye! Have a nice day.",
    "exit": "Goodbye! Have a nice day.",
    "thanks": "You're welcome!",
    "how are you": "I'm doing great!",
    "help": "Try saying Hi, Hello, Who are you, Bye, etc."
}
```

To ensure processing continuity, the program executes a synchronous loop. Control flags are embedded to safely handle lifecycle termination when state-ending markers are read.

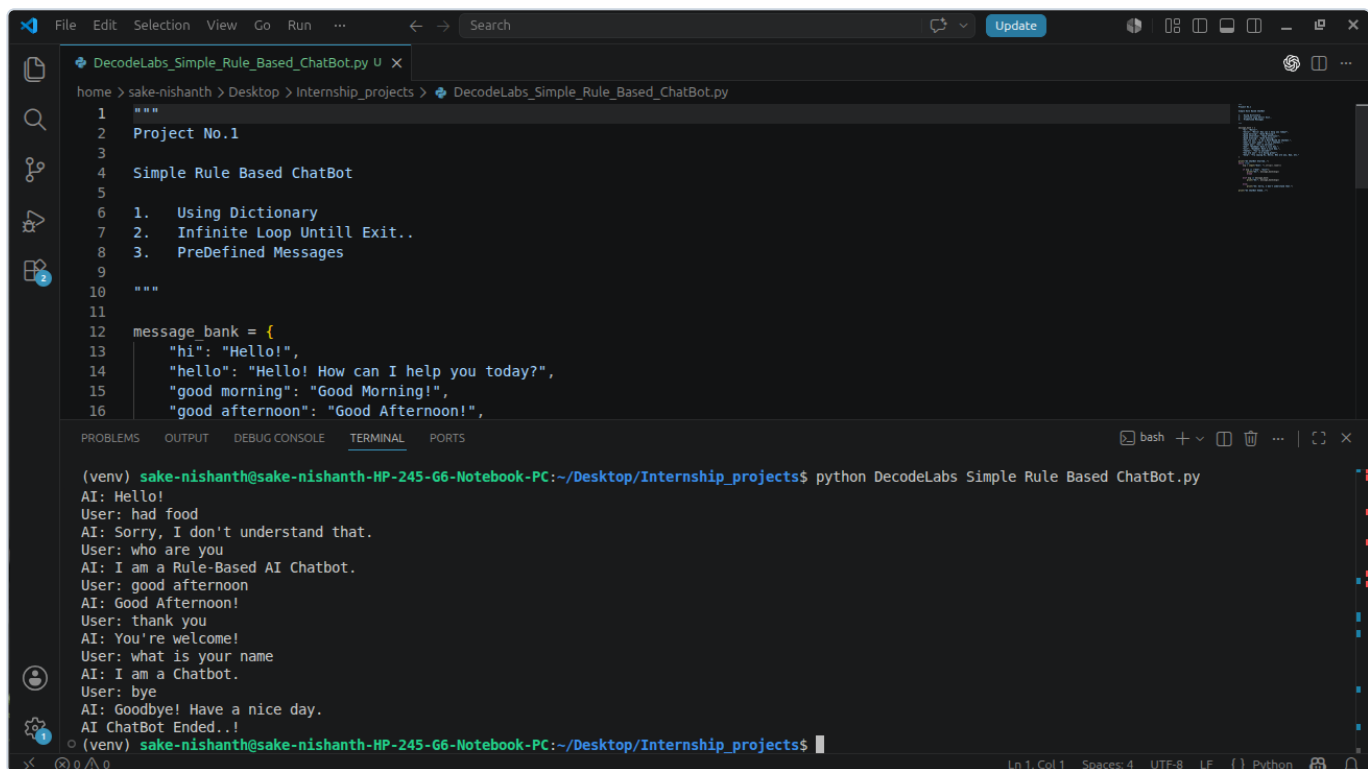
```
print("AI ChatBot Started..")
while True:
    msg = input("User: ").strip().lower()

    if msg in ["bye", "exit"]:
        print("AI:", message_bank[msg])
        break
    elif msg in message_bank:
        print("AI:", message_bank[msg])
    else:
        print("AI: Sorry, I don't understand that.")

print("AI ChatBot Ended..!")
```

### 4. Execution & Execution Environment Validation

The execution logs indicate structural accuracy under standard conversational inputs and edge-case exceptions. The system successfully normalizes inputs by converting casing, thereby satisfying token matching protocols. When presented with out-of-vocabulary (OOV) tokens such as "had food", the fallback mechanism triggers gracefully without system failure.



The image shows a Visual Studio Code editor window. The top pane displays a Python script named `DecodeLabs_Simple_Rule_Based_ChatBot.py`. The script contains a dictionary `message_bank` with keys for "hi", "hello", "good morning", and "good afternoon". The bottom pane shows the terminal output of the script, which is a simple rule-based chatbot. The chatbot responds to "Hello!" with "Hello!", "had food" with "Sorry, I don't understand that.", "who are you" with "I am a Rule-Based AI Chatbot.", "good afternoon" with "Good Afternoon!", "thank you" with "You're welcome!", "what is your name" with "I am a Chatbot.", "bye" with "Goodbye! Have a nice day.", and "AI ChatBot Ended..!".

```
1 """
2 Project No.1
3
4 Simple Rule Based ChatBot
5
6 1. Using Dictionary
7 2. Infinite Loop Untill Exit..
8 3. PreDefined Messages
9
10 """
11
12 message_bank = {
13     "hi": "Hello!",
14     "hello": "Hello! How can I help you today?",
15     "good morning": "Good Morning!",
16     "good afternoon": "Good Afternoon!",
17 }
```

```
(venv) sake-nishanth@sake-nishanth-HP-245-G6-Notebook-PC:~/Desktop/Internship_projects$ python DecodeLabs Simple Rule Based ChatBot.py
AI: Hello!
User: had food
AI: Sorry, I don't understand that.
User: who are you
AI: I am a Rule-Based AI Chatbot.
User: good afternoon
AI: Good Afternoon!
User: thank you
AI: You're welcome!
User: what is your name
AI: I am a Chatbot.
User: bye
AI: Goodbye! Have a nice day.
AI ChatBot Ended..!
(venv) sake-nishanth@sake-nishanth-HP-245-G6-Notebook-PC:~/Desktop/Internship_projects$
```

Figure 1.1: Runtime terminal output showing processing states, data normalization, and fallback execution.

## 5. Evaluation of Limitations & Future Scope

While rule-based mechanisms ensure low latency and high reliability, they exhibit high fragility under semantic variations. If a user types *"Could you please assist me?"* instead of the exact token *"help"*, the hash lookup fails.

Future system layers can mitigate this by integrating regular expression matrices (Regex), Levenshtein Distance algorithms to handle typographical variance, or moving towards vector embeddings to map conversational intents across multi-dimensional semantic spaces.